

torch.fft.irfft#

<https://pytorch.org/docs/stable/generated/torch.fft.irfft.html>

Description:

Computes the inverse of `rfft()`. input is interpreted as a one-sided Hermitian signal in the Fourier domain, as produced by `rfft()`. By the Hermitian property, the output will be real-valued. Some input frequencies must be real-valued to satisfy the Hermitian property. In these cases the imaginary component will be ignored. For example, any imaginary component in the zero-frequency term cannot be represented in a real output and so will always be ignored.

Function Signature

```
torch.fft.irfft(input, n=None, dim=-1, norm=None, *,  
out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> t = torch.linspace(0, 1, 5)
>>> t
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
>>> T = torch.fft.rfft(t)
>>> T
tensor([ 2.5000+0.0000j, -0.6250+0.8602j, -0.6250+0.2031j])
```

```
>>> torch.fft.irfft(T)
tensor([0.1562, 0.3511, 0.7812, 1.2114])
```

```
>>> roundtrip = torch.fft.irfft(T, t.numel())
>>> torch.testing.assert_close(roundtrip, t, check_stride=False)
```

Notes & Warnings

NOTE

Note Some input frequencies must be real-valued to satisfy the Hermitian property. In these cases the imaginary component will be ignored. For example, any imaginary component in the zero-frequency term cannot be represented in a real output and so will always be ignored.

NOTE

Note The correct interpretation of the Hermitian input depends on the length of the original data, as given by n . This is because each input shape could correspond to either an odd or even length signal. By default, the signal is assumed to be even length and odd signals will not round-trip properly. So, it is recommended to always pass the signal length n .

NOTE

Note Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension. With default arguments, size of the transformed dimension should be $(2^n + 1)$ as argument n defaults to even
$$\text{output size} = 2 * (\text{transformed_dim_size} - 1)$$

Detailed Documentation

Documentation

Computes the inverse of `rfft()`.

input is interpreted as a one-sided Hermitian signal in the Fourier domain, as produced by `rfft()`. By the Hermitian property, the output will be real-valued.

Some input frequencies must be real-valued to satisfy the Hermitian property. In these cases the imaginary component will be ignored. For example, any imaginary component in the zero-frequency term cannot be represented in a real output and so will always be ignored.

The correct interpretation of the Hermitian input depends on the length of the original data, as given by `n`. This is because each input shape could correspond to either an odd or even length signal. By default, the signal is assumed to be even length and odd signals will not round-trip properly. So, it is recommended to always pass the signal length `n`.

Supports `torch.half` and `torch.chalf` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimension. With default arguments, size of the transformed dimension should be $(2^n + 1)$ as argument `n` defaults to even output size = $2 * (\text{transformed_dim_size} - 1)$

`input (Tensor)` – the input tensor representing a half-Hermitian signal

`n (int, optional)` – Output signal length. This determines the length of the output signal. If given, the input will either be zero-padded or trimmed to this length before computing the real IFFT. Defaults to even output: $n=2*(\text{input.size(dim)} - 1)$.

`dim (int, optional)` – The dimension along which to take the one dimensional real IFFT.

`norm (str, optional)` –

Normalization mode. For the backward transform (`irfft()`), these correspond to:

"forward" - no normalization

"backward" - normalize by $1/n$

"ortho" - normalize by $1/\sqrt{n}$ (making the real IFFT orthonormal)

Calling the forward transform (`rfft()`) with the same normalization mode will apply an overall normalization of $1/n$ between the two transforms. This is required to make `irfft()` the exact inverse.

Default is "backward" (normalize by $1/n$).

`out` (Tensor, optional) – the output tensor.

Without specifying the output length to `irfft()`, the output will not round-trip properly because the input is odd-length:

So, it is recommended to always pass the signal length `n`: